AWS VPC Peering

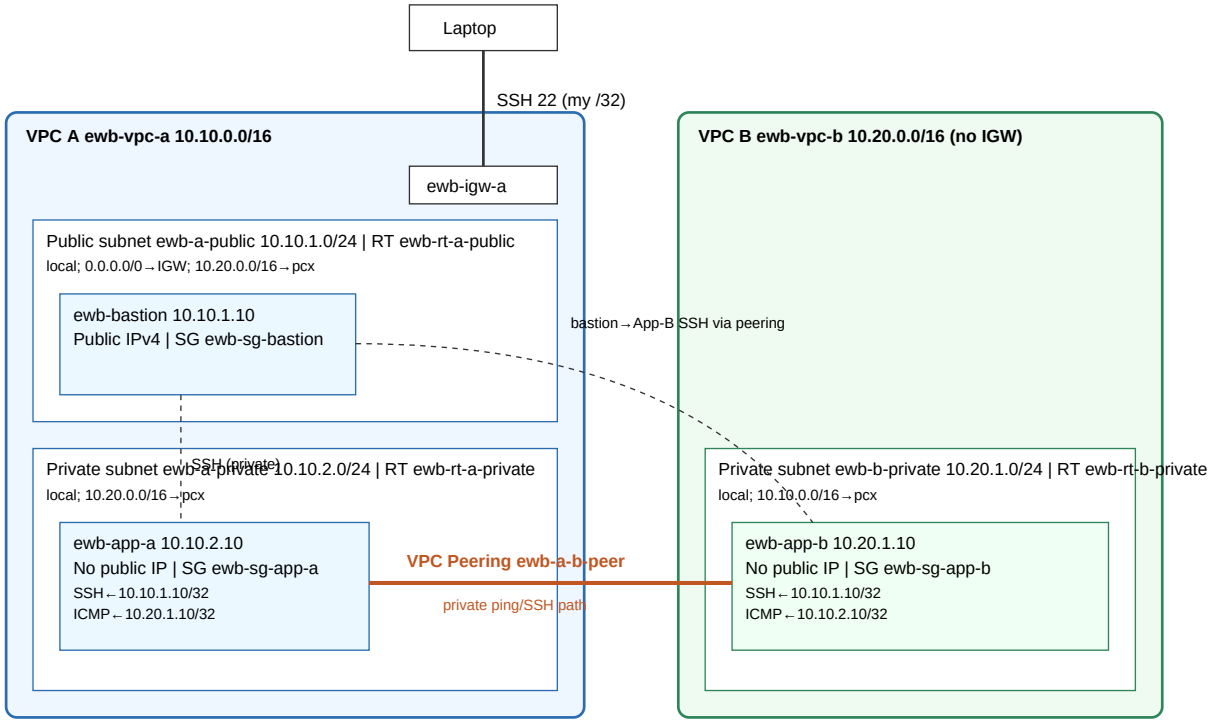
2 VPCs · 3 EC2 Instances · 1 Private VPC Peering Connection

Name	Prince Kumar
College	National Institute of Advanced Manufacturing Technology (NIAMT), Ranchi
Course	B.Tech in Computer Science and Engineering
Project Title	AWS VPC Peering
Project Type	AWS Cloud Networking Project
Organization	EWB Courses
Email	princekumar2005itaha@gmail.com
Report Date	October 2026

1. Project Objective

EWB has an application in VPC A and a second workload in VPC B. Both application servers must stay private. The objective is to create a VPC peering connection so they can exchange traffic using private IPv4 addresses, with one public bastion host in VPC A for administration. The project covers non-overlapping CIDR planning, public and private subnets with explicit route table associations, peering with return routes, least-privilege security groups, SSH access through ProxyJump, and a test method that includes a baseline failure, success tests and a controlled failure. All resources are created in a single AWS account and a single Region, with no NAT Gateway.

2. Architecture



Administration path: laptop → bastion → App-A or App-B using SSH ProxyJump. Project traffic: App-A ↔ App-B over private IPs through the peering connection. Peering does not give VPC B internet access through VPC A.

3. Network Design and Address Plan

Resource / Name tag	VPC or subnet	IPv4 CIDR / address
ewb-vpc-a	VPC A	10.10.0.0/16
ewb-a-public	VPC A / public subnet	10.10.1.0/24
ewb-a-private	VPC A / private subnet	10.10.2.0/24
ewb-vpc-b	VPC B	10.20.0.0/16
ewb-b-private	VPC B / private subnet	10.20.1.0/24
ewb-bastion	ewb-a-public	10.10.1.10 + public IPv4
ewb-app-a	ewb-a-private	10.10.2.10 / no public IP
ewb-app-b	ewb-b-private	10.20.1.10 / no public IP

Other resources: Internet gateway **ewb-igw-a**; route tables **ewb-rt-a-public, ewb-rt-a-private, ewb-rt-b-private**; security groups **ewb-sg-bastion, ewb-sg-app-a, ewb-sg-app-b**; peering connection **ewb-a-b-peer**; key pair **ewb-lab**. All resources are tagged Project = EWB-VPC-Peering and Student = Prince Kumar.

Why the CIDRs do not overlap: 10.10.0.0/16 spans 10.10.0.0–10.10.255.255 and 10.20.0.0/16 spans 10.20.0.0–10.20.255.255. The ranges are disjoint, so each destination IP belongs to exactly one VPC and peering routes are unambiguous. Each /24 subnet lies inside its own VPC range. Different subnet ranges alone cannot fix overlapping VPC CIDRs, and AWS refuses to peer overlapping VPCs.

4. VPC A and VPC B

VPC A (ewb-vpc-a, 10.10.0.0/16) is created with the “VPC only” option so every subnet and route is controlled manually. It contains the public subnet ewb-a-public (10.10.1.0/24) and the private subnet ewb-a-private (10.10.2.0/24), both in the same Availability Zone. The Internet Gateway ewb-igw-a is attached to VPC A. The public route table ewb-rt-a-public is explicitly associated with ewb-a-public and holds the local route plus 0.0.0.0/0 → ewb-igw-a. The private route table ewb-rt-a-private is explicitly associated with ewb-a-private and initially holds only the local route; no internet default route is added, and auto-assign public IPv4 stays disabled on the private subnet.

VPC B (ewb-vpc-b, 10.20.0.0/16) contains only the private subnet ewb-b-private (10.20.1.0/24), explicitly associated with ewb-rt-b-private (local route only initially). VPC B has no Internet Gateway and public IPv4 auto-assignment is disabled.

5. EC2 Instances

Three instances use Amazon Linux 2023 and a small x86 instance type (t3.micro or the instructor-approved type) with a small root volume set to delete on termination. One RSA key pair, **ewb-lab**, is used for all instances; the private .pem key remains only on the local laptop and is never uploaded to any instance or repository. Private IPs are assigned at launch.

Instance	Network settings	Security group
ewb-bastion	VPC A / ewb-a-public; private IP 10.10.1.10; public IPv4 enabled	ewb-sg-bastion
ewb-app-a	VPC A / ewb-a-private; private IP 10.10.2.10; no public IPv4	ewb-sg-app-a
ewb-app-b	VPC B / ewb-b-private; private IP 10.20.1.10; no public IPv4	ewb-sg-app-b

6. Security Groups

Security group	Type / port	Source
ewb-sg-bastion (VPC A)	SSH / TCP 22	Current public IPv4 of the laptop /32
ewb-sg-app-a (VPC A)	SSH / TCP 22	10.10.1.10/32 (bastion private IP)
ewb-sg-app-a (VPC A)	ICMP IPv4 Echo request (type 8, code 0)	10.20.1.10/32 (App-B)
ewb-sg-app-b (VPC B)	SSH / TCP 22	10.10.1.10/32 (bastion private IP)
ewb-sg-app-b (VPC B)	ICMP IPv4 Echo request (type 8, code 0)	10.10.2.10/32 (App-A)

Outbound rules stay at the default allow-all and network ACLs stay at the default allow-all. Security groups are stateful, so replies to permitted traffic are allowed automatically. **/32 rules** permit exactly one host (least privilege). SSH and ICMP are never exposed to 0.0.0.0/0. **Bastion access:** the bastion is the only internet-facing instance and accepts SSH only from my own public IP. App-A and App-B accept SSH only from the bastion's private IP 10.10.1.10, because the bastion opens a separate SSH connection to them (to App-B over the peering), so App-B sees the bastion's private address as the source.

7. VPC Peering

Field	Value
Name	ewb-a-b-peer
Requester	ewb-vpc-a (10.10.0.0/16)
Accepter	ewb-vpc-b (10.20.0.0/16)
Account / Region	Same AWS account / same Region

The request is created from VPC A and must be accepted (Actions → Accept request) even though both VPCs belong to the same account. The connection must reach the Active status before routes are added. Peering is only the link: routes and security groups decide whether traffic can use it.

8. Routing

After peering is Active, the missing routes are added to all three route tables (pcx-ID is the actual ewb-a-b-peer connection; no unnecessary routes are created):

Route table / association	Destination	Target	Purpose
ewb-rt-a-private → ewb-a-private	10.10.0.0/16 10.20.0.0/16	local pcx-ID	VPC A route App-A to VPC B
ewb-rt-b-private → ewb-b-private	10.20.0.0/16 10.10.0.0/16	local pcx-ID	VPC B route Return path to VPC A
ewb-rt-a-public → ewb-a-public	10.10.0.0/16 0.0.0.0/0 10.20.0.0/16	local ewb-igw-a pcx-ID	Bastion to App-A Bastion internet access Bastion SSH to App-B

Forward path: App-A (10.10.2.10) sends traffic for 10.20.1.10; ewb-rt-a-private matches 10.20.0.0/16 and forwards it to the peering connection, which delivers it to App-B.
Return path: App-B replies to 10.10.2.10; ewb-rt-b-private matches 10.10.0.0/16 and sends the reply back through the peering connection. Without this return route, replies are dropped and ping fails even when peering is Active.
Public route table: ProxyJump makes the bastion open its own connection to App-B, so its subnet must know how to reach VPC B. The more specific 10.20.0.0/16 route is selected over the 0.0.0.0/0 internet route.

9. SSH Access with ProxyJump

The key is stored at ~/.ssh/ewb-lab.pem with chmod 400 (WSL, macOS or Linux). The local ~/.ssh/config contains the entries below, where BASTION_PUBLIC_IP is the bastion's current public IPv4:

```
Host ewb-bastion
    HostName BASTION_PUBLIC_IP
    User ec2-user
    IdentityFile ~/.ssh/ewb-lab.pem
    IdentitiesOnly yes

Host ewb-app-a
    HostName 10.10.2.10
    User ec2-user
    IdentityFile ~/.ssh/ewb-lab.pem
    IdentitiesOnly yes
    ProxyJump ewb-bastion

Host ewb-app-b
    HostName 10.20.1.10
    User ec2-user
    IdentityFile ~/.ssh/ewb-lab.pem
    IdentitiesOnly yes
    ProxyJump ewb-bastion
```

ssh ewb-bastion and ssh ewb-app-a work before peering through VPC A's local route; ssh ewb-app-b works only after the peering and routes are complete. This lets administration reach private hosts without copying any private key onto an instance.

10. Testing Methodology

Verification uses real terminal output from the instances. The table lists each test, where it is run and the acceptance criteria.

Test	Where / command	Expected result
01 Baseline failure	App-A, before peering and peer routes: ping -c 4 10.20.1.10	No replies (100% packet loss)
02 Forward path	App-A, after routes: ping -c 4 10.20.1.10	4 replies; 0% packet loss
03 Reverse path	App-B, after routes: ping -c 4 10.10.2.10	4 replies; 0% packet loss
04 Private SSH	Laptop: ssh ewb-app-b	Shell on App-B through the bastion
05 Isolation	EC2 instance details of App-A and App-B (Public IPv4 empty); hostname -I on each host	No public IPv4; all tests use private destination IPs
06 Controlled failure	Remove only the App-B inbound ICMP Echo request rule from 10.10.2.10/32; ping from App-A	Ping fails; SSH rule remains intact
06 Recovery	Restore the exact ICMP rule; rerun ping from App-A	4 replies; 0% packet loss

Commands run inside each private host: `hostname`, `hostname -I`, `ping -c 4 PEER_PRIVATE_IP`.

Baseline failure: before peering, no route to 10.20.0.0/16 exists in ewb-rt-a-private, so the ping gets no reply, which proves isolation. **Successful connectivity:** once peering is Active with routes on both sides and the Echo request rules in place, ping works in both directions. **Controlled failure and recovery:** removing only the ICMP rule on App-B breaks ping while peering and routes stay unchanged, which shows that Active peering alone does not guarantee reachability and that security groups also control the path; restoring the rule recovers the connectivity.

11. Key AWS Networking Concepts

• Peering is one-to-one and non-transitive: A-B plus A-C does not create B-C connectivity.
• Routes are required on both sides; a one-way route is insufficient.
• An Active peering does not guarantee ping or SSH; route tables and security groups decide.
• VPC CIDRs must not overlap; subnet ranges alone cannot fix overlap.
• VPC B cannot use VPC A’s Internet Gateway over peering, and needs no Internet Gateway.
• Security groups are stateful; /32 sources allow only specific hosts.
• A public subnet needs an IGW route, and the instance needs a public IPv4 for direct internet access.
• Private instances have no internet egress here, as no NAT Gateway is used; a failed package update does not mean peering is broken.

12. Troubleshooting

Symptom	What to inspect
Bastion SSH times out	Bastion's current public IP; my current public IP /32 in the SSH rule; public subnet association; 0.0.0.0/0 route to the attached IGW
App-A SSH fails	App-A private IP; TCP 22 allowed from 10.10.1.10/32; correct key, ec2-user and running instance
App-B SSH fails	Bastion public subnet needs 10.20.0.0/16 → pcx-ID; VPC B needs the return route; App-B SSH source is the bastion private IP
Ping fails after acceptance	Both private route tables; correct pcx-ID and Active state; Echo request rule on the destination; actual host IPs
Peering creation fails	Check every CIDR (including secondary CIDRs) of both VPCs for overlap
Permission denied (publickey)	Correct key pair, local IdentityFile and ec2-user; routing works if SSH authentication was reached
Failure with custom ACLs	Check both directions, ICMP rules and SSH return ephemeral ports; this lab uses default allow-all ACLs

13. Viva Questions and Answers

Why is the return route needed?
Replies must find a path back. VPC B needs 10.10.0.0/16 → pcx, otherwise responses are dropped.

Why is the App-B SSH source the bastion private IP?
The bastion opens its own SSH connection to App-B over the peering, so App-B sees the bastion's private IP 10.10.1.10 as the source, not the laptop.

Why does Active peering not guarantee successful ping?
Active only means the link exists; routes, security groups and NACLs must also allow the traffic.

Can VPC B use VPC A's internet gateway over peering?
No. Peering does not allow edge-to-edge routing through another VPC's IGW.

Why must VPC CIDRs not overlap?
Overlap makes routing ambiguous, so AWS does not allow such VPCs to be peered.

Why is the bastion public while App-A/App-B remain private?
The bastion is the single controlled entry point; the application servers stay unreachable from the internet.

Why is a /32 source used in security groups?
It allows exactly one host (least privilege).

Why does the public route table need a VPC-B peering route?
The bastion sits in the public subnet and must reach App-B for ProxyJump; the specific 10.20.0.0/16 route wins over 0.0.0.0/0.

Is VPC peering transitive?
No. It is one-to-one only.

Why does VPC B not need an Internet Gateway?
All its traffic is private through peering, so no IGW is needed and exposure is reduced.

14. Cleanup

Evidence is saved first and only project resources are removed, in this order:

1. Terminate ewb-bastion, ewb-app-a and ewb-app-b; confirm their EBS volumes are deleted and remove any leftover volumes or snapshots.
2. Release any Elastic IP created (none is needed in this design).
3. Delete the lab key pair ewb-lab and the local key file if no longer needed.
4. Delete the peer routes and the peering connection ewb-a-b-peer.
5. Delete the security groups after the instance network interfaces disappear.
6. Remove the subnet associations, then delete the three subnets and the three custom route tables.
7. Detach and delete ewb-igw-a.
8. Delete both VPCs.
9. Verify by the project tag that no EC2, volume, public IPv4 allocation or VPC resource remains, and check Billing for remaining chargeable resources.

15. Learning Summary

This project taught me how to plan non-overlapping VPC and subnet CIDRs, build public and private subnets with explicit route table associations, and use an Internet Gateway only where needed. I learned that a VPC peering connection needs routes on both sides and correct security-group rules before traffic flows, that Active peering does not guarantee reachability, and that peering is non-transitive. I also learned bastion-host design with SSH ProxyJump so private instances can be administered without copying private keys, applying least privilege with /32 security-group rules, using a controlled failure to find which control blocks traffic, and cleaning up resources to avoid unnecessary charges.